PROJECT TITLE	:	HTML QUICK STYLER
TEAM ID	:	69a92a102bcee27bd8d0f20c
TEAM MEMBERS	:	RISHA HARINI S SRIVARSHINI E THANUSHA R SHARON LAVINA TERESA L
COLLEGE NAME	:	S. A. V SAHAYA THAI ARTS AND SCIENCE (WOMEN) COLLEGE
COLLEGE CODE	:	138
DATE	:	09.03.2026

HTML Quick-Styler

AI-Powered HTML Page Generator with IBM Granite 3.3 2B

PROJECT DESCRIPTION

HTML Quick-Styler is an intelligent web page generation system that helps users create modern HTML websites quickly using Artificial Intelligence. The system uses the IBM Granite 3.3 2B Instruct model along with Python and Gradio to generate complete HTML page structures based on user inputs such as website title, description, and color theme.

The main objective of this project is to simplify website development for beginners, students, and developers. Instead of writing HTML and CSS code manually, users can enter simple details and the system automatically generates a complete, production-ready website layout.

The application integrates AI capabilities with a user-friendly interface. Users can generate websites using two methods: a template-based generator and an AI-powered generator. The template generator quickly creates structured web pages with predefined sections such as hero section, features section, about section, portfolio gallery, testimonials, pricing, contact form, and footer. The AI generator uses the IBM Granite model to dynamically generate HTML code based on the user description.

The project also includes a color palette system that allows users to choose from eight different visual themes: Modern, Vibrant, Ocean, Forest, Sunset, Luxury, Creative, and Minimal. These palettes automatically control the background color, primary color, accent color, and text color of the generated website.

By combining AI technology with modern web development practices, HTML Quick-Styler provides a fast and easy solution for creating visually appealing websites without requiring advanced coding skills.

SCENARIOS

Scenario 1: Portfolio Website Generation

A student or freelancer wants to create a personal portfolio website to showcase their work. Instead of designing the entire page manually, the user enters the website title and description in the HTML Quick-Styler interface.

The system automatically generates a website containing a hero section that displays the user's name and introduction, a features section that highlights skills or services, a portfolio gallery to display work samples, and a footer section providing copyright information.

This allows the user to quickly generate a professional portfolio page without spending much time on coding or design.

Scenario 2: Startup Landing Page

A startup company wants to create a landing page to promote their product. The user enters the product name as the website title and provides a short description explaining the product features and benefits.

The system generates a modern webpage containing a hero section with a call-to-action button, feature blocks describing the product benefits, pricing plans, testimonials, and a footer section. By selecting an appropriate color palette such as Vibrant or Modern, the startup can easily create a visually attractive landing page.

This helps businesses quickly launch product pages without complex development processes or hiring a web developer.

Scenario 3: Educational Website

Teachers or educational institutions may want to create a simple webpage to present course information or study material. Using HTML Quick-Styler, the teacher enters the course title and description of the topics to be covered.

ARCHITECTURE OVERVIEW

The HTML Quick-Styler system is designed using a modular architecture that combines Python programming, AI model integration, and web interface development.

Python is used as the core programming language for implementing the application logic. The system loads the IBM Granite 3.3 2B Instruct model using the Hugging Face Transformers library to enable AI-based HTML code generation.

The application interface is developed using Gradio, which allows users to interact with the system through a simple web interface. Users can input website details, select color palettes, and preview generated web pages directly in the browser without any additional setup.

The system includes four main modules. The Color Palette Generator module manages eight different design themes and provides color variables for the CSS engine. The HTML Page Builder module constructs the HTML structure based on detected keywords in the user description. The CSS Styling Engine generates a complete responsive stylesheet using the selected color palette. The HTML Generation Engine coordinates all modules to assemble the final complete HTML document.

The modular design makes the system easy to maintain and extend with additional features in the future.

Core Technologies

- Python 3.x: Core programming language for application logic
- IBM Granite 3.3 2B Instruct: AI model for intelligent HTML content generation
- Gradio: Interactive web interface framework for user interaction
- HTML5/CSS3: Modern web standards for page generation
- PyTorch: Deep learning framework used for model inference
- Google Colab: Cloud-based development and deployment environment
- Responsive Design: Mobile-first approach with CSS media queries

PRE-REQUISITES

Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- Gradio: Install via pip (pip install gradio)
- Transformers & PyTorch: Install via pip (pip install transformers torch accelerate)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and model access

Knowledge Prerequisites

To understand the project implementation, basic knowledge of the following topics is useful:

- Python programming fundamentals
- HTML page structure and CSS styling basics
- Basic understanding of AI and language models
- Familiarity with Google Colab environment
- Object-oriented programming concepts in Python

Hardware Requirements

The hardware requirements for this project are minimal because most of the processing is performed in the cloud environment. Minimum system requirements include:

- Processor: Intel i3 or equivalent
- RAM: Minimum 4 GB
- Storage: At least 1 GB free space
- Display: 1280 x 720 resolution or higher
- Stable internet connection for model loading

PROJECT WORKFLOW

The development of the HTML Quick-Styler system follows a structured workflow that includes environment setup, core development, interface implementation, deployment, and testing. Each phase focuses on building specific components of the application.

The workflow ensures that the system is developed in a modular and organized manner, making the application easier to maintain, test, and extend in the future.

The main phases of the project include:

- Phase 1: Environment Setup and Model Configuration
- Phase 2: Core Application Development
- Phase 3: Gradio Interface Development
- Phase 4: Deployment
- Phase 5: Testing and Validation

Phase 1: Environment Setup & Model Configuration

This phase focuses on preparing the development environment and configuring the AI model required for the project. The application is developed using Google Colab, which provides a cloud-based environment with GPU acceleration for running Python code and AI models.

Activity 1.1: Install Dependencies and Setup Environment

Before implementing the application, necessary Python packages must be installed. These packages provide functionalities for AI model execution, web interface development, and performance optimization. The required libraries are:

- Gradio
- Transformers
- Accelerate
- Torch (PyTorch)

Activity 1.2: Configure IBM Granite Model

In this step, the IBM Granite 3.3 2B Instruct model is loaded using the Transformers library with AutoTokenizer and AutoModelForCausalLM. The model is loaded with

float16 precision and automatic device mapping to optimize memory usage. The tokenizer converts input text into tokens that the model can process, and the model then generates output based on the given prompt. This configuration ensures efficient performance when generating HTML content.

```
# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

print("Loading IBM Granite model... (this takes 3-5 minutes the first time)")

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained(
    "ibm-granite/granite-3.3-2b-instruct",
    torch_dtype=torch.float16,
    device_map="auto"
)

print("✅ Model loaded successfully!")
```

```
Loading IBM Granite model... (this takes 3-5 minutes the first time)
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%|| 787/787 [00:00<00:00, 37.9kB/s]
tokenizer_config.json: 9.93k/? [00:00<00:00, 198kB/s]
vocab.json: 777k/? [00:00<00:00, 14.7MB/s]
merges.txt: 442k/? [00:00<00:00, 7.13MB/s]
```

Phase 2: Core Application Development

This phase focuses on building the main components of the system including color palette management, HTML page generation, and CSS styling. These components work together to generate complete web pages based on user inputs.

Activity 2.1: Color Palette Generator

The system includes a ColorPaletteGenerator class that allows users to select from eight different visual themes for their websites. Each palette contains eight predefined color values controlling background, primary, secondary, accent, surface, text, and border colors. The available palettes are: Modern, Vibrant, Ocean, Forest, Sunset, Luxury, Creative, and Minimal. When a user selects a palette, the corresponding color values are automatically applied to the generated HTML using CSS custom properties.

Activity 2.2: HTML Page Builder

The HTMLPageBuilder class is responsible for generating the HTML structure of the website. It contains three key static methods. The extract_sections method scans the user description for keywords to identify which sections to include such as hero, features, about, portfolio, testimonials, pricing, contact, and footer. The generate_meta_tags method creates SEO-optimized meta tags from the page title and

description. The generate_section_html method builds the HTML markup for each individual section of the page.

Activity 2.3: CSS Styling Engine

To improve the visual appearance of the generated website, the system includes a CSS styling engine built into the HTMLPageBuilder class. The CSS rules define the layout, fonts, colors, spacing, and animations used in the webpage. Styling features include a center-aligned hero section, styled buttons with hover effects, feature cards with glassmorphism shadows, responsive layout using CSS Grid, smooth fade-in and slide animations, and full mobile responsiveness with media queries for screens below 768px.

```python
# ============================================================

class ColorPaletteGenerator:
    """Generate visually cohesive color palettes"""

    @staticmethod
    def generate_palette(style: str = "modern") -> Dict[str, str]:
        """Generate a cohesive color palette based on style"""

        palettes = {
            'modern': {
                'primary': '#667eea',
                'secondary': '#764ba2',
                'accent': '#f093fb',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'vibrant': {
                'primary': '#ff6b6b',
                'secondary': '#ff8e72',
                'accent': '#ffd93d',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
```

Phase 3: Gradio Interface Development

The HTML Quick-Styler application uses Gradio Blocks to build an interactive multi-tab user interface. Gradio allows developers to create web-based interfaces for machine learning applications quickly. The interface is organized into five tabs.

Tab 1: Generate Page

This tab allows users to generate websites using predefined templates. Users enter the website title, description of the desired sections, and select a color style. Once the Generate HTML button is clicked, the system produces a complete HTML page and displays the generated code along with a color palette summary and status message.

Tab 2: Live Preview

This tab renders the generated HTML directly in the browser window, allowing users to see the final visual result of the generated page without needing to open a separate file or browser tab.

Tab 3: Color Palettes

This tab allows users to explore and compare all eight available color themes before generating their page. Users can select any style and view a detailed preview of the primary, secondary, accent, background, and text colors included in that palette.

Tab 4: Templates

This tab provides three pre-configured ready-made templates for rapid one-click generation: Portfolio Website, SaaS Landing Page, and Company Website. Each template auto-fills the title, description, and color style fields, allowing users to generate a full page instantly.

Tab 5: Guide

This tab provides a step-by-step user guide explaining how to use the tool, which keywords trigger which sections, how to preview the generated page, and how to save the HTML file for use in a browser.

```python
class HTMLPageBuilder:
    """Build styled HTML pages from descriptions"""

    @staticmethod
    def extract_sections(description: str) -> List[str]:
        """Extract sections from description"""
        sections = []

        section_keywords = {
            'hero': ['hero', 'banner', 'landing'],
            'features': ['features', 'services', 'benefits'],
            'about': ['about', 'team', 'company'],
            'portfolio': ['portfolio', 'gallery', 'work', 'projects'],
            'testimonials': ['testimonial', 'reviews', 'feedback'],
            'pricing': ['pricing', 'plans', 'cost'],
            'contact': ['contact', 'form', 'reach'],
            'footer': ['footer', 'bottom']
        }

        desc_lower = description.lower()
        for section, keywords in section_keywords.items():
            if any(kw in desc_lower for kw in keywords):
                sections.append(section)

        if not sections:
            sections = ['hero', 'features', 'about', 'contact', 'footer']

        return sections
```

Phase 4: Deployment

After implementing all system components, the application is deployed using Google Colab and Gradio sharing functionality. Gradio automatically generates a public shareable link that allows users to access the application from any web browser without installing additional software. The deployment command uses demo.launch(share=True) which makes the application accessible from anywhere with an internet connection.

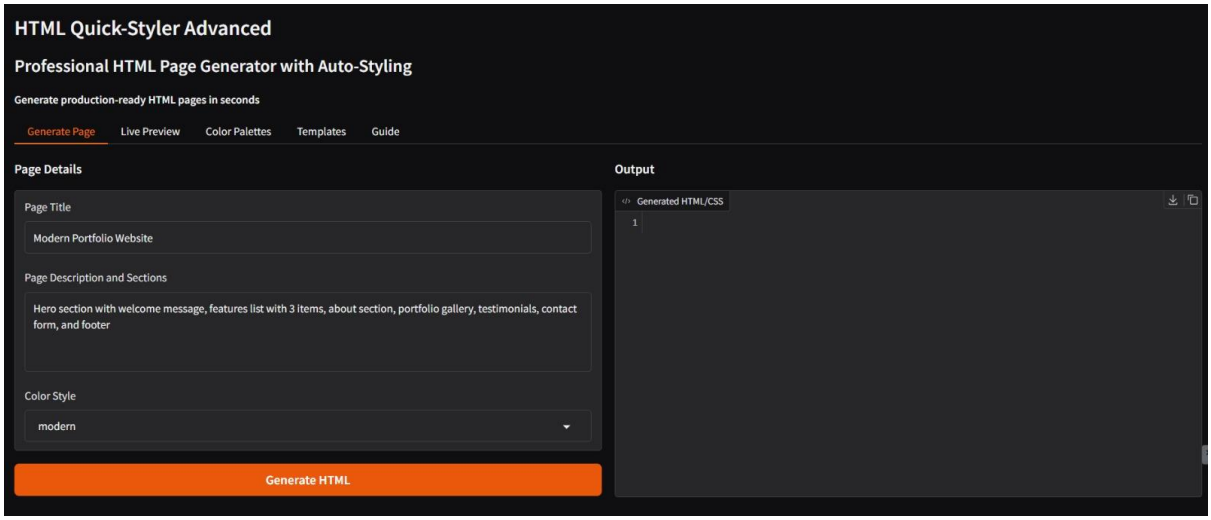
```python
class HTMLGenerationEngine:
    """Main engine for HTML page generation"""

    @staticmethod
    def generate_page(title: str, description: str, style: str = "modern") -> Tuple[str, str]:
        """Generate complete HTML page"""

        # Generate color palette
        palette = ColorPaletteGenerator.generate_palette(style)

        # Extract sections
        sections = HTMLPageBuilder.extract_sections(description)

        # Generate meta tags
        meta_tags = HTMLPageBuilder.generate_meta_tags(title, description)

        # Generate CSS
        css = HTMLPageBuilder.generate_css(palette)

        # Generate sections HTML
        sections_html = '\n\n'.join([
            HTMLPageBuilder.generate_section_html(section, palette)
            for section in sections
            if HTMLPageBuilder.generate_section_html(section, palette)
        ])

        # Build complete HTML
        html = f'''<!DOCTYPE html>
```

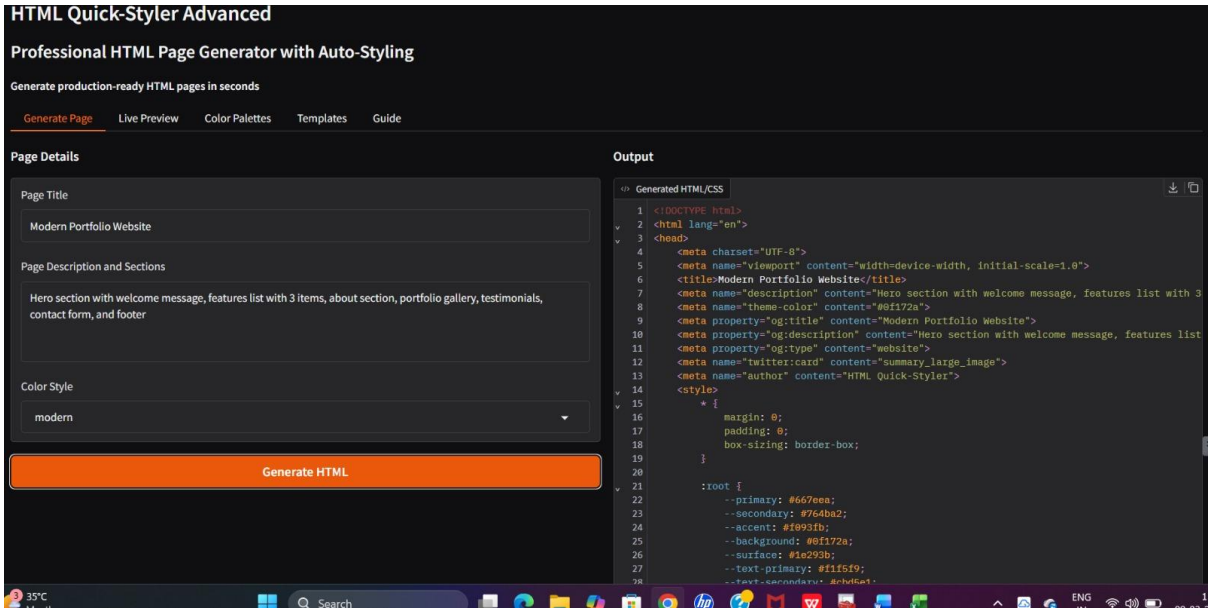
Terminal

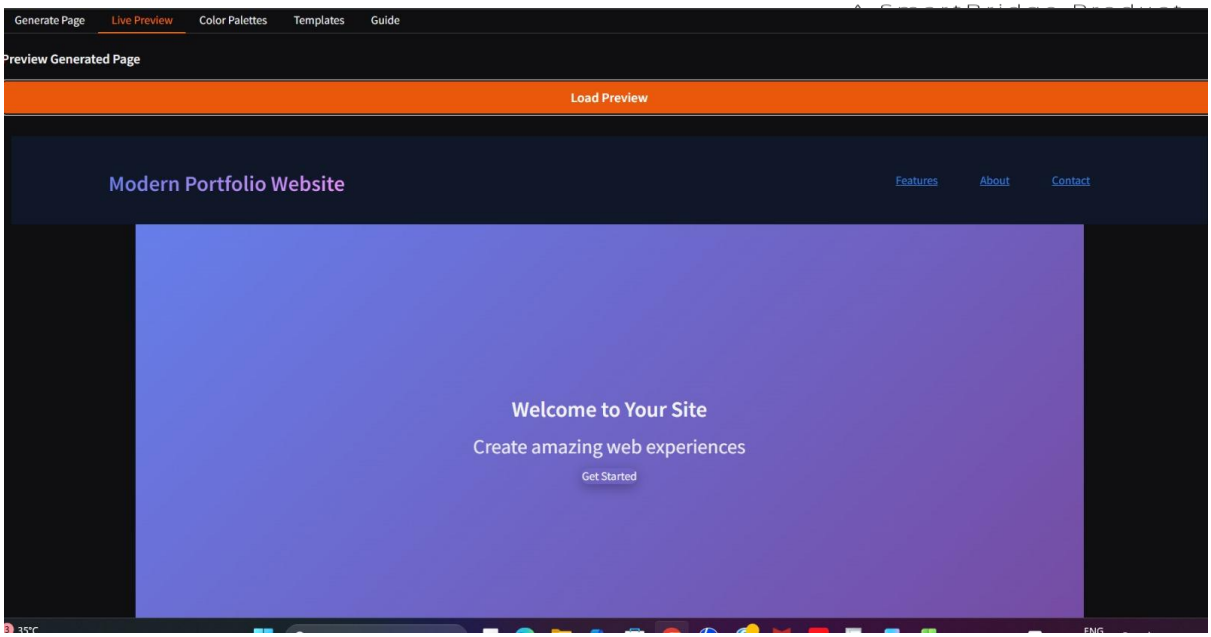
Phase 5: Testing and Validation

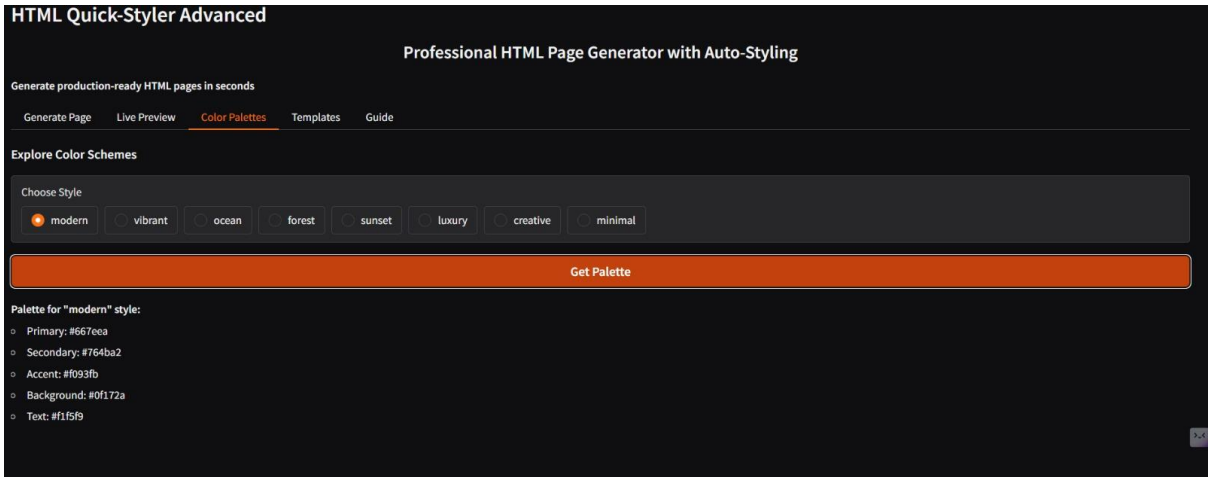
Testing is performed to ensure that the system works correctly and generates valid HTML pages. Functional testing verifies that user inputs are accepted correctly, HTML templates are generated successfully, color palettes are applied properly, the interface displays the generated webpage correctly, and all five tabs function as expected. Different input values including various titles, descriptions, and color styles are tested to confirm that the system behaves consistently across all scenarios.

Output:


HTML Quick-Styler Advanced
Professional HTML Page Generator with Auto-Styling
Generate production-ready HTML pages in seconds
Generate Page
Live Preview
Color Palettes
Templates
Guide
Page Details
Page Title
Modern Portfolio Website
Page Description and Sections
Hero section with welcome message, features list with 3 items, about section, portfolio gallery, testimonials, contact form, and footer
Color Style
modern
Generate HTML
Output
Generated HTML/CSS


HTML Quick-Styler Advanced
Professional HTML Page Generator with Auto-Styling
Generate production-ready HTML pages in seconds
Generate Page
Live Preview
Color Palettes
Templates
Guide
Page Details
Page Title
Modern Portfolio Website
Page Description and Sections
Hero section with welcome message, features list with 3 items, about section, portfolio gallery, testimonials, contact form, and footer
Color Style
modern
Generate HTML
Output
Generated HTML/CSS
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Modern Portfolio Website</title>
<meta name="description" content="Hero section with welcome message, features list with 3
<meta name="theme-color" content="#0f172a">
<meta property="og:title" content="Modern Portfolio Website">
<meta property="og:description" content="Hero section with welcome message, features list
<meta property="og:type" content="website">
<meta name="twitter:card" content="summary_large_image">
<meta name="author" content="HTML Quick-Styler">
<style>
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}
:root {
--primary: #667eea;
--secondary: #764ba2;
--accent: #f093fb;
--background: #0f172a;
--surface: #1e293b;
--text-primary: #f1f5f9;


Generate Page
Live Preview
Color Palettes
Templates
Guide
Preview Generated Page
Load Preview
Modern Portfolio Website
Features
About
Contact
Welcome to Your Site
Create amazing web experiences
Get Started


HTML Quick-Styler Advanced
Professional HTML Page Generator with Auto-Styling
Generate production-ready HTML pages in seconds
Generate Page
Live Preview
Color Palettes
Templates
Guide
Explore Color Schemes
Choose Style
modern
vibrant
ocean
forest
sunset
luxury
creative
minimal
Get Palette
Palette for "modern" style:
Primary: #667eea
Secondary: #764ba2
Accent: #f093fb
Background: #0f172a
Text: #f1f5f9

HTML Quick-Styler Advanced

Professional HTML Page Generator with Auto-Styling

Generate production-ready HTML pages in seconds

Generate Page | Live Preview | Color Palettes | Templates | Guide

HTML Quick-Styler Guide

How It Works

1. Enter Details

- Title: Page name
- Description: Sections you want
- Color Style: Pick a theme

2. Automatic Detection

The tool recognizes these keywords:

- hero or banner = Hero Section
- features or services = Features Grid
- about or team = About Section
- portfolio or gallery = Portfolio
- testimonial or reviews = Testimonials
- pricing or plans = Pricing Plans
- contact or form = Contact Form
- footer = Footer

3. Generate and Preview

CONCLUSION

The HTML Quick-Styler project demonstrates how artificial intelligence can simplify the process of creating websites. By integrating the IBM Granite 3.3 2B AI model with Python and Gradio, the system enables users to generate modern, production-ready HTML webpages quickly and efficiently without requiring any manual coding.

The application successfully combines template-based generation and AI-powered generation to provide flexibility for different types of users. Beginners can use the pre-built templates to create simple websites instantly, while advanced users can experiment with the AI-powered generator and customize layouts using detailed descriptions.

The eight-theme color palette system adds significant customization options that improve the visual appearance of generated webpages while maintaining consistency throughout the design. The modular class-based architecture ensures that the system is clean, maintainable, and easy to extend with additional features.

The Gradio multi-tab interface provides an intuitive experience that guides users from input to live preview in a few simple steps. The responsive CSS output ensures that all generated pages are mobile-friendly by default.

Overall, the HTML Quick-Styler project successfully shows how AI tools can assist developers, designers, and non-technical users in building web applications faster while maintaining clean, structured, and standards-compliant code.